BPFScript: Compiler-Checked Cross-Boundary Types for eBPF Applications

Paper ID: TBD

Abstract

A realistic eBPF application spans several coupled artifacts: a verifier-constrained kernel program, a userspace loader, shared maps, and generated skeletons. The relationships across them (which program a handle refers to, a map's key and value type on both sides of the kernel boundary, and the load-to-attach-to-detach order) are usually implicit naming conventions checked only at verifier-load or attach time. We study BPFScript, a typed DSL that lifts these relationships into the type system: typed maps, typed program-lifecycle handles, and execution domains turn cross-boundary mismatches into compiler diagnostics while still lowering to ordinary C through the stock clang/bpftool/libbpf toolchain. On the artifact at commit 3b19cd2, a suite of 27 invalid programs confirms that each invariant fires; 43 of 44 examples compile and 41 build into libbpf-compatible projects; and the generated objects verifier-load, attach, pass traffic, and run a tcp-congestion struct_ops object on a real kernel through a direct libbpf runner. The result is a concrete design point for typed cross-boundary eBPF structure that preserves the kernel toolchain while exposing its remaining compatibility boundaries.

CCS Concepts: • **Software and its engineering** → **Domain specific languages**; *Static analysis*; • **Computer systems organization** → *Reliability*.

Keywords: eBPF, domain-specific language, static typing, systems

1 Introduction

eBPF lets Linux developers run application-specific logic inside the kernel under a verifier that proves memory safety and bounded execution [18]. It now underpins packet processing, traffic control, tracing, security policy, and scheduler experimentation. Yet the programming model is awkward for a structural reason: a single eBPF *application* is split across artifacts that must agree with one another. The kernel program is written under verifier constraints; a userspace coordinator opens objects, finds maps, installs links, and tears resources down; maps and ring buffers carry shared state; BTF describes kernel types; and newer interfaces such as kfuncs and struct_ops require generated skeleton code to match the installed kernel headers.

The reference C/libbpf workflow is powerful but verbose [15]: the developer hand-writes kernel C, userspace C, map setup, attachment and pinning logic, cleanup paths, and a Makefile. Libraries in Rust, Go, or Python improve this experience, and some (notably Aya) author both sides from one host-language source [1, 6]; tracing languages such as bpftrace raise the abstraction level for one domain [3]. But even when both sides are authored together, the relationships that span the kernel boundary, lifecycle ordering and domain separation, stay largely runtime or convention rather than compiler-checked types.

The key observation is that these relationships are single concepts duplicated across artifacts. A map declaration is at once a kernel data structure, a userspace lookup target, and a type contract; a program reference is at once a function name, BPF section, skeleton field, and file descriptor. In C/libbpf, those copies are tied together by naming conventions and generated headers, so mismatches surface late. In one typed source, the compiler can see and check them directly.

This paper studies that idea through BPFScript, a compiled, multi-domain DSL in which a developer writes eBPF entry points, helpers, userspace control flow, maps, ring buffers, perf events, and optional kernel-module code in one source language. The compiler lowers this source to eBPF C, userspace C, an optional module, and a Makefile, after which the ordinary libbpf and bpftool path produces objects, skeletons, and binaries. BPFScript is beta software; its value here is as a prototype for studying the design space, not as a libbpf replacement.

This paper makes three contributions. *(1)* We formulate BPFScript's central idea as a *typed model of cross-boundary eBPF structure*: execution domains, typed maps shared across the kernel boundary, and a program lifecycle modeled as typed producer/consumer handles, each specified as a typing rule with a matching compiler diagnostic (Section 3). *(2)* We show how the prototype realizes that model while preserving the stock kernel toolchain, so compatibility failures remain visible and measurable rather than hidden (Section 4). *(3)* We give a reproducible artifact evaluation (Section 5) that confirms each typed invariant fires, that one source generates buildable libbpf projects, and that the generated objects load, attach, and run on a real kernel, while being explicit about the limits of that evidence. All scripts, logs, derived tables, and paper macros live outside the cloned BPFScript tree so the evaluation can be rerun without editing the source under test.

2 Background: Three Couplings

An eBPF application has at least two components, a restricted kernel program and a userspace coordinator, communicating

through maps, ring buffers, perf events, or global data. This split creates three kinds of coupling that a language could, in principle, check.

Lifecycle coupling. Operations must occur in a valid order: an object is opened, loaded, attached, read, and detached using type-appropriate APIs. "Attach before load" or "read a handle that was never produced" are ordering errors that C/libbpf catches only at runtime.

Representation coupling. The same concept (a map, a typed context field) must be expressed consistently in kernel C, generated skeleton headers, userspace lookup code, and BTF-derived declarations. A key or value type that disagrees across those copies is a silent mismatch until something fails.

Compatibility coupling. eBPF interfaces evolve quickly. struct_ops, kfuncs, dynptrs, and scheduler extensions depend on the exact kernel, bpftool, and libbpf versions, so generated skeletons can disagree with installed headers.

The verifier is essential but deliberately late. A developer may write, compile, generate skeletons, link a loader, and only then receive a rejection. A DSL helps if it moves the first two couplings earlier, into type errors, without hiding the execution model or abandoning the toolchain that enforces the third.

3 A Typed Model of Cross-Boundary Structure

This section sketches the parts of BPFScript's type system that encode the couplings of Section 2. We describe the model informally rather than giving a full semantics or soundness proof. Every rule corresponds to a diagnostic exercised by the rejection suite of Section 5 (parenthesized names are the cases in `experiments/static_checks/`).

A BPFScript program is a flat sequence of declarations. Listing 1 shows the minimal shape of an application; the rest of this section makes its three relationships precise.

Listing 1. A unified-source eBPF application.

```
include "xdp.kh"

@xdp fn pass_all(ctx: *xdp_md) -> xdp_action {
  return XDP_PASS
}

fn main() -> i32 {
  var prog = load(pass_all)
  attach(prog, "lo", 0)
  detach(prog)
  return 0
}
```

Domains as attributes (representation coupling). A function attribute assigns each function to an execution domain $d \in \{user, ebpf, helper, kfunc\}$. @xdp, @tc, and @probe mark eBPF entry points. So do @tracepoint and @perf_event. @helper marks kernel-shared eBPF code; @kfunc and @private mark kernel-module code; an unattributed fn is userspace. The checker tags each function with its domain and rejects illegal crossings, for example a userspace function calling an @helper (helper_from_userspace) or requesting kernel-context allocation from a userspace or XDP domain (gfp_from_userspace, gfp_from_xdp). Each eBPF attribute also fixes a context and return type: @xdp requires ctx:*xdp_md returning xdp_action; @tc requires *__sk_buff returning i32. A signature that disagrees with its attribute is rejected before code generation (xdp_wrong_context, tc_wrong_context, xdp_wrong_return, probe_wrong_return). The same surface syntax therefore remains domain-aware: userspace can allocate and print freely, while eBPF code must obey stack and helper restrictions. struct_ops callbacks fit the same scheme, with context and return types determined by the registered slot.

Maps as typed globals (representation coupling). A map is a typed global, varm:hash<K,V>(N) or array<K,V>(N), rather than a C section plus separate userspace lookup code. Its element types form one contract checked at every access site on *both* sides of the kernel boundary: an index m[k] requires k:K and yields V, in eBPF and userspace alike. A wrong key type, wrong value type, or access to an undeclared map is rejected (map_string_key, map_string_value, map_undefined). One declaration thus replaces kernel definitions, skeleton fields, and userspace lookups that otherwise must agree by hand.

Program lifecycle as typed handles (lifecycle coupling). The most protocol-like cross-boundary invariant is lifecycle. BPFScript distinguishes a FunctionRef (a reference to an @-attributed function) from a ProgramHandle (a loaded program), and types the lifecycle builtins so that only load produces a ProgramHandle. The formation rule for FunctionRef is also the rule that ties lifecycle to domains: only an eBPF entry-point function inhabits FunctionRef, so an @helper, @kfunc, or userspace function is simply not a value load can accept.

$$\frac{f \text{ has an eBPF entry attribute}}{\Gamma \vdash f : \texttt{FunctionRef}}$$

$$\frac{\Gamma \vdash f : \texttt{FunctionRef}}{\Gamma \vdash \texttt{load}(f) : \texttt{ProgramHandle}}$$

$$\frac{\Gamma \vdash h : \texttt{ProgramHandle}}{\Gamma \vdash \texttt{detach}(h) : \texttt{unit}}$$

$$\frac{\Gamma \vdash h : \texttt{ProgramHandle}}{\Gamma \vdash \texttt{attach}(h, t, fl) : \texttt{u32}}$$

Because attach and detach consume a ProgramHandle and only load produces one, "attach before load" is a *type* error, not a runtime failure. Passing a bare FunctionRef (attach_without_load), an integer (attach_integer_handle), or a string (load_string, detach_string) to a lifecycle builtin does not type-check. This is a producer/consumer handle discipline, not yet a full typestate automaton: because only

load produces a ProgramHandle, it enforces the single ordering constraint $load \prec \{attach, detach\}$ rather than the complete load-to-attach-to-detach protocol. It guarantees that lifecycle builtins receive values of the right kind, but not full path-sensitive properties such as use-after-detach, double-attach, or leaked handles.

From invariants to diagnostics. The payoff is that cross-boundary mismatches become localized compiler errors rather than naming conventions or late verifier/attach failures. Section 5 measures whether those diagnostics fire; Section 6 explains the limits of that evidence.

4 Implementation

The evaluated compiler is written in OCaml with dune. It parses BPFScript source, resolves imports and includes, builds symbol tables, runs multi-program analysis and type checking, performs safety analysis, builds an IR, and emits C. For an input `foo.ks` the generated project normally contains `foo.ebpf.c`, `foo.c`, a `Makefile`, and, when kfuncs are present, `foo.mod.c`. The Makefile uses bpftool to dump `vmlinux.h`, clang to produce a BPF object, bpftool to generate a skeleton, and gcc to link the userspace loader against libbpf, libelf, and zlib.

Two implementation choices follow directly from the model. First, the compiler has *separate modules* for maps, userspace codegen, eBPF codegen, kernel-module generation, context-specific codegen, safety checking, BTF parsing, dynptr bridging, tail-call analysis, and struct_ops; it encodes eBPF concepts directly rather than acting as a thin preprocessor over C, which is what gives the type checker somewhere to attach domain-specific checks. Second, the compiler *preserves the kernel toolchain*: it does not replace the verifier, clang, bpftool, or libbpf, so it inherits BTF, skeleton generation, and existing diagnostics. Crucially, any compatibility failure remains visible and measurable instead of being papered over. For interfaces that move faster than the DSL can stabilize, `include` and `extern` declarations provide an escape hatch to generated header declarations.

Two lowerings illustrate the same idea beyond the immediate kernel/userspace split. Source-level ring-buffer operations (reserve, field write, submit) lower to the appropriate dynptr helpers, so the author never hand-assembles that lifetime. A userspace stage may also exec() into Python and reuse open map descriptors: the loader clears FD_CLOEXEC, passes descriptor numbers through the environment, and the compiler emits a Python wrapper that reopens typed access from compile-time metadata (test_exec.ks). The map schema and descriptor handoff still come from one typed source rather than hand-written glue.

5 Evaluation

We evaluate BPFScript as a compiler and systems artifact, asking three questions that track the contributions: *(Q1)* does each typed cross-boundary invariant actually fire as a compile-time diagnostic? *(Q2)* does one typed source generate a complete, buildable libbpf project? *(Q3)* are the generated artifacts real, that is, do they load, attach, and run on a kernel rather than merely compile?

Setup. The evaluation runs on Linux 6.15.11-061511-generic with clang 18, bpftool 7.7, libbpf 1.3.0, OCaml 4.14, and dune 3.14, on a Intel(R) Core(TM) Ultra 9 285K with 24 CPUs, at commit 3b19cd2. The harness writes raw stdout and stderr for every compiler and make invocation, per-example CSV rows, per-script JSON, and the numeric macros used in this paper, so every aggregate traces back to a logged row rather than a hand-edited table. SLOC counts are nonblank, noncomment lines; generated `vmlinux.h` and skeleton headers are excluded from expansion factors because they are bpftool outputs, not BPFScript outputs. Timing numbers below are local, unpinned samples (CPU governor powersave, turbo enabled, no CPU pinning, virtualization none) reported as sanity evidence rather than performance claims; we report medians with ranges.

5.1 Q1: The typed invariants fire

Regression coverage. The compiler test suite reports 85 successful suites and 1095 passing tests with no failures, covering lexer/parser, type checking, maps and map assignment, userspace codegen, context-field typing, safety, ring buffers, perf-event attach, detach APIs, kfunc attributes, struct_ops, and BTF parsing, indicating that the implementation covers the domain concepts it claims to encode rather than only a small example set.

Static rejection suite. The central Q1 result is a suite that exercises the diagnostics of Section 3 directly. The harness compiles 28 targeted programs and verifies that every outcome matches expectation: one positive control plus 27 expected rejections: 5 lifecycle API misuses, 5 program-signature violations, 3 map type/symbol errors, 4 general type errors, 2 symbol-validation errors, 1 config-boundary violation, 1 helper-scope violation, 2 kernel-context allocation violations, 2 perf-event group-bound violations, 1 ring-buffer API misuse, and 1 stack-limit violation. These cases line up one-to-one with the lifecycle, map, and domain rules of Section 3. The oracle is stronger than a nonzero exit code: each case carries an expected diagnostic substring, so incidental rejections do not count as passing. The suite is therefore a *diagnostic-contract regression test*: it shows that the invariants are implemented, mapped to the intended error classes, and stable across compiler changes. It does not by itself show how often those checks catch real mistakes; the next paragraph probes that question.

Differential failure-stage probe. To ask whether the checks fire *earlier* than the stock toolchain on the same mistake, we pre-register 4 realistic mistakes, each injected identically into a working XDP map-counter written in BPFScript and in matched hand-written C/eBPF, and record the earliest

Table 1. Differential failure-stage probe: earliest stage each toolchain catches the same injected mistake. BPFScript is strictly earlier only on the cross-boundary context-type mistake; the rest are ties clang already catches.

Injected mistake	Coupling	KS	C/libbpf
Wrong context type	signature/domain	compile reject	undetected
Undeclared map	representation	compile reject	compile reject
String into `u64` value	representation	compile reject	compile reject
Oversized stack buffer	safety	compile reject	compile reject

stage each toolchain rejects it along a shared ladder (compile < build < verifier < attach < runtime < undetected). Table 1 reports the outcome with the kernel verifier live (verifier tested: yes). BPFScript rejects all 4 at compile time. The decisive case is the cross-boundary one: an `@xdp` entry point handed a `*__sk_buff` context instead of `*xdp_md`. BPFScript rejects it from the program-signature rule of Section 3, whereas clang compiles it and the verifier *accepts* it, so in C the mistake survives to a silently wrong load. The other 3 mistakes tie because clang already catches them locally: an undeclared map, a string stored into a `u64` value, and an oversized stack buffer. The result is therefore 1 earlier and 3 tied: typing helps precisely on the cross-boundary relation that local C checking cannot see. Section 6 explains the limits of this small, author-injected set.

5.2 Q2: One source generates the project

Buildability and centralization. Of 44 examples, 43 compile from BPFScript source (97.7%) and 41 build into generated C/eBPF projects (93.2%). The single compile-time rejection is `safety_demo.ks`, where safety analysis detects 608 bytes of stack use, above the 512-byte eBPF limit, and halts before codegen, a direct instance of a verifier-relevant problem caught early. For successful examples the compiler emits a median 472 lines from a median 31 BPFScript source lines (excluding `vmlinux.h` and skeletons); this 11.3x ratio measures emitted boilerplate, not effort saved. A closer proxy is a matched comparison: across 11 workloads with hand-written C/libbpf baselines, the BPFScript sources total 203 lines versus 1105 (5.44x; median 4.6x per workload). Both are author-written corpora, and Section 6 reports the advantage does not survive an independent port. These examples exercise many domains: Table 2 shows attempted, compiler-accepted, and fully built counts across XDP, TC, probes, tracepoints, perf, maps, ring buffers, dynptr, kfunc, struct_ops, tail calls, and userspace, showing that the artifact spans more than tracing while making the struct_ops gap explicit.

Table 2. Example marker coverage. A = attempted, KS = BPFScript compiler success, B = generated project build success.

Feature	A	KS	B	Feature	A	KS	B
XDP	38	37	37	TC	5	5	5
Probe	4	4	4	Tracept.	2	2	2
Perf	2	2	2	Maps	17	16	16
Ringbuf	3	3	3	Dynptr	1	1	1
kfunc	4	4	3	Str._ops	2	2	0
Tail	2	2	2	User	39	39	37

Does the centralization transfer? The expansion factor is measured on a self-authored corpus, so we test it against code the BPFScript authors did not write. We hand-ported 3 pinned `xdp-tutorial` XDP programs to BPFScript. The ports total 45 nonblank noncomment lines while the original external C/eBPF program bodies total 45 lines. This near-parity is the most important number in the external check: on independent code the large self-authored ratios do *not* reproduce. The comparison is not perfectly matched: BPFScript includes the userspace `main`, while the external count relies on `xdp-tutorial`'s shared loader. The right reading is therefore modest: centralization is corpus-dependent and must be measured per workload. For external feature context, a source-only scan of 3 pinned public eBPF repositories (166 files) observes 14 of the tracked feature families, confirming that the local program classes appear in real code.

5.3 Q3: The generated artifacts are real

Build success is weaker than kernel acceptance, which is weaker than attachability, which is weaker than running. Table 3 summarizes this progression on a real kernel.

Table 3. Real-kernel checks for generated artifacts: compile, build, verifier-load, and attach.

Stage	Population	Pass
BPFScript compile	44	43
Generated project build	44	41
Verifier-load (strict)	41	37
Isolated XDP attach/detach	27	27

The verifier matrix script loads generated objects with bpftoolprogloadall. It counts success only when bpftool returns success *and* at least one BPF program is pinned, avoiding false positives from empty sections. Of the 41 objects from fully built projects, 37 verifier-load (90.2%). The 4 failures split into one reference-ownership leak, one map-creation failure, one missing kfunc symbol in kernel BTF, and one object with no program pinned. The attach matrix script then takes the verifier-clean single-section XDP subset, attaches each object to a fresh veth in its own

network namespace, confirms prog/xdp in ip-dlinkshow, and detaches it. 27 of 27 attach and detach cleanly (100.0%). The attach matrix covers XDP; TC and struct_ops are exercised by the workloads below, while probes, tracepoints, perf events, and ring buffers are verifier-loaded but not attach-tested here. The "runs on a real kernel" claim is therefore scoped to XDP, TC, and struct_ops.

Runtime sanity. Beyond load and attach, matched workloads confirm the generated objects *run*. Under BPF_PROG_TEST_RUN, a minimal generated XDP pass program matches its hand-written baseline in both instruction count and median latency (2 instructions, 5 (5–6) vs. 6 (5–6)). A map-counter program does not: the generated object emits 21 instructions versus 11 for C (12 (12–13) vs. 9 (8–9)). That comparison is not matched because the C baseline uses an atomic add while the generated path lowers to a separate lookup and update. This gap reflects a backend lowering choice rather than the typed model itself. Over 10 veth/iperf3 trials, generated and hand-written XDP pass programs deliver comparable local medians (17.4 (16.0–18.8) vs. 17.8 (14.8–19.2) Gb/s), and matched TC programs pass traffic as well. These are local, unpinned veth/TCP samples that show the generated object runs without collapsing the data path.

Where the toolchain leaks through. Preserving the kernel toolchain means BPFScript also inherits its compatibility coupling, and the artifact makes this visible rather than hiding it. The two generated build failures (sched_ext_simple.ks, struct_ops_simple.ks) are not parser or type errors: both compile to eBPF objects and generate skeletons. But bpftool v7.7.0 emits a generated skeleton that assigns the field bpf_map_skeleton.link; the older installed libbpf 1.3.0 headers on our host do not provide that field (4.5% of examples). This is a mismatched-toolchain pairing, with newer bpftool against older libbpf, not an inherent property of either version. A matched bpftool/libbpf pair would not exhibit it. To separate this skeleton-header skew from object compatibility, a shared libbpf runner loads both the generated and a matched hand-written tcp-congestion struct_ops object *without* generated skeletons. On this host (bpftool v7.7.0, libbpf 1.3.0, skeleton map-link support no) both objects load, attach, and detach in 3/3 and 3/3 trials. The generated object is therefore valid; the failure is a version-skew problem in the generated *skeleton*, exactly the compatibility coupling of Section 2 that source unification does not eliminate.

6 Limitations and Discussion

The evaluation supports a bounded reading of the contribution and clarifies what remains open.

The organic probe is small. Section 5 injects only 4 realistic mistakes into matched BPFScript and C/eBPF examples, finding BPFScript strictly earlier on 1 and tied on 3. That establishes a preview, not a population estimate. A stronger study would mine real eBPF mistakes from revision histories and issue trackers, port them to BPFScript, and report the resulting failure stages.

Lifecycle typing is lightweight. We enforce only the producer/consumer handle distinction, not full flow-sensitive typestate, so use-after-detach, double-attach, and leaked handles are not statically ruled out on every path.

The corpus is author-written. The expansion and feature results come from the BPFScript repository, which reflects features the authors chose to demonstrate; the external port shows the line-count advantage does not transfer unchanged. A stronger study would reimplement programs from Cilium, libbpf-tools, and production filters in both BPFScript and C/libbpf.

No runtime-equivalence claim. Traffic and microbenchmark numbers are local, unpinned veth/TCP samples that establish sanity, not performance. A deployment study would run each example in a VM with a known kernel configuration, representative workloads, and explicit post-detach cleanup checks.

Toolchain version alignment matters. The struct_ops result is a mismatched bpftool/libbpf pairing rather than a deep limitation, but it shows that a generated build depends on alignment between kernel headers, bpftool, generated skeletons, and libbpf. A production BPFScript should record the target kernel ABI, check libbpf feature support before generation, and fail early with a dependency diagnostic.

The Python bridge is a mechanism, not yet a result. The `exec()`-to-Python path lets the compiler, not the developer, carry typed maps across the process boundary, but the checked-in example stops at direct map reads and writes: it is a usable path into the Python ecosystem, not a machine-learning integration result.

7 Related Work

C/libbpf and host-language bindings. XDP established the in-kernel programmable packet model BPFScript targets [9]; broader surveys cover the surrounding landscape [18, 20]. C/libbpf is the reference workflow but leaves kernel code, loaders, skeletons, maps, and build logic as separate artifacts [15]; scaffolding such as libbpf-bootstrap and xdp-tutorial cuts the boilerplate yet leaves the cross-boundary relationships untyped [14, 19]. Host-language libraries go part of the way: Aya in particular compiles both sides from one Rust source and shares typed maps across the boundary [1, 6], so single-source authoring and typed maps are not by themselves BPFScript's contribution. BPFScript differs in being host-language-independent, with execution domains and their crossing rules in the language. It lowers to the standard Linux BPF toolchain rather than a host runtime, so BTF, CO-RE, skeletons, and verifier diagnostics are inherited and compatibility skew stays measurable. Rex similarly narrows the language-verifier gap with safe Rust

extensions, but like Aya targets a host language and its own loading path rather than stock libbpf [12].

Tracing and bytecode tools. bpftrace is concise but narrows the target to tracing [3]. BCC scripts eBPF from Python but keeps Python as a controller around separate BPF code [11]; related systems use eBPF to *observe* distributed LLM inference [5] or move the runtime itself to userspace [23]. A separate line optimizes the eBPF bytecode after it exists: PREVAIL [8], K2 [22], and hXDP [4]; BPFScript is earlier and complementary, generating the bytecode and its loader.

Dataplane DSLs and typestate. Compiling one typed description into a split low-level implementation has precedent: P4 compiles a protocol-independent program into target parser and match-action pipelines [2], and Click composes packet processing from typed elements [13]. Unlike a P4 pipeline, an eBPF application must also generate its userspace coordinator and survive a late general-purpose verifier and fast-moving libbpf/bpftool ABIs, the source of the compatibility limits we measure. BPFScript's lifecycle discipline is a lightweight instance of typestate [17]: we enforce only the producer/consumer handle distinction, so the connection is a design influence rather than a claim of equivalent guarantees. The unrealized guarantees are substructural: an affine `ProgramHandle` consumed by `detach` (or a typed `Link` returned by `attach`) would rule out use-after-detach, double-attach, and leaks [21], Vault enforces such resource protocols in low-level software [7], and session types formalize the ordering more fully [10]. Our domain separation echoes address-space type qualifiers such as the kernel's Sparse `__user`/`__kernel` annotations [16]. BPFScript thus occupies a point not covered above: a compiled, multi-domain, single-source language whose question is whether eBPF application structure can be made explicit enough for a compiler to type and generate the low-level project *while preserving* the existing Linux BPF toolchain.

8 Conclusion

BPFScript treats the cross-boundary structure of an eBPF application (program lifecycle, shared maps, and execution domains) as something a compiler can type rather than something developers hold together by convention and discover at verifier-load or attach time. On the checked-in artifact, a suite of 27 invalid programs confirms each typed invariant fires as a localized diagnostic; 43 of 44 examples compile and 41 build from one source into libbpf-compatible projects; and the generated objects verifier-load, attach, pass traffic, and run a tcp-congestion struct_ops object on a real kernel via a direct libbpf runner. The result is a concrete, reproducible design point: typing eBPF's cross-boundary structure is feasible, and because the prototype lowers through the stock toolchain, both what the type system buys and the version-skew limits it cannot absorb stay visible and measurable.

References

[1] Aya contributors. 2026. Aya: Rust eBPF library. https://github.com/aya-rs/aya.

[2] Pat Bosshart, Dan Daly, Glen Gibb, Martin Izzard, Nick McKeown, Jennifer Rexford, Cole Schlesinger, Dan Talayco, Amin Vahdat, George Varghese, and David Walker. 2014. P4: Programming Protocol-Independent Packet Processors. *ACM SIGCOMM Computer Communication Review* 44, 3 (2014), 87–95.

[3] bpftrace developers. 2026. bpftrace: High-level tracing language for Linux eBPF. https://github.com/bpftrace/bpftrace.

[4] Marco Spaziani Brunella, Giacomo Belocchi, Marco Bonola, Salvatore Pontarelli, Giuseppe Siracusano, Giuseppe Bianchi, Aniello Cammarano, Alessandro Palumbo, Luca Petrucci, and Roberto Bifulco. 2020. hXDP: Efficient Software Packet Processing on FPGA NICs. In *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 973–990.

[5] Kexin Chu, Jianchang Su, Yifan Zhang, Chenxingyu Zhao, Yiwei Yang, Yusheng Zheng, Shengkai Lin, Shizhen Zhao, and Wei Zhang. 2025. eInfer: Unlocking Fine-Grained Tracing for Distributed LLM Inference with eBPF. In *Proceedings of the 3rd Workshop on eBPF and Kernel Extensions*. 76–83. doi:10.1145/3748355.3748372

[6] Cilium contributors. 2026. cilium/ebpf: eBPF library for Go. https://github.com/cilium/ebpf.

[7] Robert DeLine and Manuel Fähndrich. 2001. Enforcing High-Level Protocols in Low-Level Software. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*.

[8] Elazar Gershuni, Nadav Amit, Arie Gurfinkel, Nina Narodytska, Jorge A. Navas, Noam Rinetzky, Leonid Ryzhyk, and Mooly Sagiv. 2019. Simple and Precise Static Analysis of Untrusted Linux Kernel Extensions. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 1069–1084.

[9] Toke Høiland-Jørgensen, Jesper Dangaard Brouer, Daniel Borkmann, John Fastabend, Tom Herbert, David Ahern, and David Miller. 2018. The eXpress Data Path: Fast Programmable Packet Processing in the Operating System Kernel. In *Proceedings of the 14th International Conference on Emerging Networking Experiments and Technologies (CoNEXT)*. 54–66.

[10] Kohei Honda, Vasco T. Vasconcelos, and Makoto Kubo. 1998. Language Primitives and Type Discipline for Structured Communication-Based Programming. In *Programming Languages and Systems (ESOP)*.

[11] iovisor/bcc developers. 2026. BCC: Tools for BPF-based Linux IO analysis, networking, monitoring, and more. https://github.com/iovisor/bcc.

[12] Jinghao Jia, Ruowen Qin, Milo Craun, Egor Lukiyanov, Ayush Bansal, Minh Phan, Michael V. Le, Hubertus Franke, Hani Jamjoom, Tianyin Xu, and Dan Williams. 2025. Rex: Closing the Language-Verifier Gap with Safe and Usable Kernel Extensions. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*.

[13] Eddie Kohler, Robert Morris, Benjie Chen, John Jannotti, and M. Frans Kaashoek. 2000. The Click Modular Router. *ACM Transactions on Computer Systems* 18, 3 (2000), 263–297.

[14] libbpf authors. [n. d.]. libbpf-bootstrap: Scaffolding for BPF Application Development. https://github.com/libbpf/libbpf-bootstrap. Accessed 2026.

[15] libbpf developers. 2026. libbpf: a BPF CO-RE userspace library. https://github.com/libbpf/libbpf.

[16] Linux kernel developers. [n. d.]. Sparse: A Semantic Parser for C. https://www.kernel.org/doc/html/latest/dev-tools/sparse.html. Accessed 2026.

[17] Robert E. Strom and Shaula Yemini. 1986. Typestate: A Programming Language Concept for Enhancing Software Reliability. *IEEE Transactions on Software Engineering* SE-12, 1 (1986), 157–171.

[18] The Linux Kernel Documentation Project. 2026. eBPF Documentation. https://docs.kernel.org/bpf/.

[19] The XDP Project. [n. d.]. xdp-tutorial: XDP Hands-On Tutorial. https://github.com/xdp-project/xdp-tutorial. Accessed 2026.

[20] Marcos A. M. Vieira, Matheus S. Castanho, Racyus D. G. Pacífico, Elerson R. S. Santos, Eduardo P. M. Câmara Juńior, and Luiz F. M. Vieira. 2020. Fast Packet Processing with eBPF and XDP: Concepts, Code, Challenges, and Applications. *Comput. Surveys* 53, 1 (2020), 1–36.

[21] Philip Wadler. 1990. Linear Types Can Change the World! In *Programming Concepts and Methods*, M. Broy and C. Jones (Eds.). North-Holland.

[22] Qiongwen Xu, Michael D. Wong, Tanvi Wagle, Srinivas Narayana, and Anirudh Sivaraman. 2021. Synthesizing Safe and Efficient Kernel Extensions for Packet Processing. In *Proceedings of the 2021 ACM SIGCOMM Conference*. 50–64.

[23] Yusheng Zheng, Tong Yu, Yiwei Yang, Yanpeng Hu, Xiaozheng Lai, Dan Williams, and Andrew Quinn. 2025. Extending Applications Safely and Efficiently. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*.